

Title: Measuring CSS-based tracking in the wild

Motivation: Traditional web and email tracking techniques rely heavily on JavaScript, cookies, and tracking pixels. However, recent research has demonstrated that CSS alone can be leveraged for user fingerprinting and tracking, even in highly restricted environments such as privacy-focused browsers and email clients. While the feasibility of CSS-based tracking has been shown in controlled environments, it remains unclear how widely these techniques are actually deployed by real-world websites and email marketing campaigns. This project aims to measure the prevalence of CSS-based tracking in the wild by analyzing websites and potentially email newsletters (if time allows).

Prior Work: Previous work in the privacy and fingerprinting space has covered a wide range of techniques to attempt to uniquely identify users. CSS has also been used in the past, I was familiar with FingerprintJS' take. Another example is creepjs, which aims to “shed light on weaknesses and privacy leaks among modern anti-fingerprinting extensions and browsers.” However, I stumbled upon “Cascading Spy Sheets” on Hackernews the other day and thought it was novel at the depth and accuracy. The paper cites Lin et Al as a demonstration of how stylistic aspects of CSS can be used for fingerprinting. Cascading Spy Sheets is more broad, and discovers things like arithmetic operations that yield inconsistent results across platforms. These papers demonstrate that CSS-based tracking is possible, by showing the ability to correctly classify various browser combinations. This is cutting edge, since Xin et al was published in May 2023 and Cascading Spy Sheets was published weeks ago!

Approach: Because of the potentially infinite scope, I want to limit this project to detecting the presence of the examples demonstrated in Cascading Spy Sheets. I will likely only get to a subset of the presented methods even, but I want to develop a few approaches to detecting fingerprinting attempts. This fingerprinting is a two step issue: 1) vulnerable features of CSS are identified, and 2) exfiltration of identifying information is possible by CSS only. The paper and associated proof of concept code does introduce a browser extension attempts to address part 2, essentially blocking all possible exfiltration paths. I want to expand that and consider what detection for part 1 looks like. I may not get great results on the top 1000+ websites, but I should at least be able to detect the provided proof of concepts and maybe tools like FingerprintJS or creepjs to the extent they use CSS. Their extension resulted in an overhead of 30%, so I am optimistic some queries will be suspicious. **At the very least, effort will have gone in to creating a test bed and an initial stab at detection approaches**, which I view as sufficient for this class! I will model the approach after classic software quality assessment techniques:

- Static analysis – look for suspicious patterns with @container, @import, @supports, @font-face, specifically look for loads inside conditional @container queries
- Dynamic analysis – look for differences in how webpages load between browsers, to the maximum extent possible with tools like Puppet and Selenium
- Assess complexity – advanced CSS arithmetic features with crazy precision may indicate malicious intent
- Potentially assess things like dependency graphs, to see if a lot of external resources are being loaded on the fly, especially by @import

This work is so novel, and the fingerprinting industry is so secretive, **I view this project as a “best effort” if that is an acceptable mindset**. I will enumerate detection strategies, the results from each of the tested webpages for each strategy, and attempt to develop a “confidence” score whether CSS-based fingerprinting is being employed. However, I do not expect a comprehensive result. The Spy Sheets paper is quite deep, and the experimental setup must have taken months of dedicated work to complete!